COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Start the next bundle

Continuation guide and readiness check

Where to begin, what to run and how to distinguish student work from reference evidence.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Start the next bundle

Package 03 expands Volume 02

The preceding Package 02 expanded Volume 1: Foundations and Your First Game. This bundle continues with **Volume 2: Mathematics for Games and Computing**, preserving Modules 06-10 and lessons 06.1-10.4 from the original manifest. Package numbering and volume numbering are different: Package 03 is not Volume 03. The next curriculum volume after this one is C++, Algorithms and Language Implementation.

This bundle supplies 20 expanded lessons, 20 runnable demonstrations, 20 coding-exercise starter/reference pairs, 100 lesson quiz questions, 25 module-review questions, 75 original written problems, ten algebra-bridge items, five integrating module projects, and Gate B. The lesson questions retain the original 60 and add 40 application questions. The 25 review questions are additional; the written problems, bridge and transfer tasks are counted separately.

The original roadmap assigns **500-800 planning hours to Volume 2**, including mathematical study, exercises, projects and review. That is not the time needed to read this handbook, a promised schedule, or an academic-credit statement. Work at the speed that produces independent understanding. Many lessons introduce topics that require several study sessions and substantial external problem practice.

Readiness and placement

Complete Volume 1 or demonstrate equivalent competence: run a saved Python file, write and call a function, iterate over lists, test boundary cases and explain a returned value. The ordinary route also requires Gate A's evidence. The original plan permits mathematics alongside early programming once the relevant coding prerequisites are met; paper-first work can begin before every integration task is ready.

Before Lesson 06.1, complete the ten algebra-bridge questions in the workbook. These are a diagnostic aid, not a validated placement exam. Repair incorrect rearrangements, division-domain mistakes and degree/radian confusion using R40 and the first lesson. Do not skip linear algebra because vectors appear familiar in an engine editor. Modules 08-10 require the preceding mathematical work, not just syntax recognition.

Open the files

Extract the entire ZIP before running code. Keep its folder layout intact. Open STUDENT_HANDBOOK.pdf for the course or COURSE.html for the searchable browser edition. Use LAB_WORKBOOK.pdf for coding contracts and problem sets. Keep ANSWER_KEY.pdf closed until after an attempt. CAPSTONE_SPEC.pdf defines the final mathematics notebook; MATH_REFERENCE.pdf is a compact formula-and-convention companion, not a replacement for derivations.

All required programs use the Python standard library. No GPU, game engine, numerical package, paid account or new hardware is required. Python 3.11+ is the selected syntax baseline; actual automated execution was performed with Python 3.13.5 on Linux. Compatibility with other operating

systems and versions is not claimed as separately tested. Existing figures are included; regenerating the optional PNG illustrations additionally needs Matplotlib.

Find a working interpreter

In a terminal opened in the extracted Game_Engineering_Package_03 folder, first check the interpreter. On macOS/Linux a common command is:

```
python3 --version
python3 environment_check.py
python3 run_example.py 06.1
```

On Windows with the Python launcher installed, use:

```
py -3 --version
py -3 environment_check.py
py -3 run_example.py 06.1
```

If the installed launcher is instead `python`, confirm that its --version output is Python 3, then use it consistently. In the remaining documents, `python` means that verified interpreter command; substitute `python3` or `py -3` as appropriate. Run shell commands in the terminal, not at Python's >>> prompt. Do not install tools by disabling security controls or copying administrator commands.

A first session of roughly 60-90 minutes

Begin with paper. Solve the bridge items, then read 06.1 and predict travel time for 36 meters at 6 meters/second. State why zero speed is outside the function's contract. Predict the output of the degree/radian example before running it. Record the actual command and output. Answer the lesson quiz without looking at the key, then compare your reasoning and repair one changed example.

When ready for implementation, open labs/student/lab_06_1.py. Replace its NotImplementedError body with your own calculation and validation. The source folder contains working references; do not import those implementations to make an unfinished student file pass. Run:

```
python check_lab.py 06.1
```

The initial checker failure is intentional: the student functions are unfinished exercises. A failure with NotImplementedError does not mean installation is broken. To verify the supplied reference separately:

```
python check_lab.py 06.1 --reference
python validate_reference.py
```

Work in small, inspectable steps

The recommended loop is derive -> predict -> implement -> test -> explain. A function checker covers a computational slice; a proof, assumption review, graph model or statistics discussion still requires written evidence. The reference implementation is in mathkit.py; files under labs/reference expose those functions to the checker. Demonstration code intentionally calls reference functions so every example is runnable before the exercise is finished.

Use outputs/your_name_for_the_run as a new directory when creating experiment evidence. The benchmark and reference-evidence generator refuse to overwrite an existing destination. Save edits and learner work separately from supplied reference evidence. No program uploads data or requires a server.

Notation and progress

In mathematical prose, x^2 denotes a square. In Python write x**2: the ^ operator is not exponentiation. A dot between vectors denotes an inner product. Multiplication order, angle units, array indexing, time horizon and filter alignment must be stated. Small floating-point differences are assessed using justified tolerances, not arbitrary rounding.

COURSE.html stores self-recorded progress locally when the browser permits storage. Export a backup after a study session; opening another copy, clearing site data or changing browsers may not preserve it. Progress records document your choices, not an independently verified credential. The PDFs and code work without browser storage.

Assessment and independent work

Each of the five lesson questions is scored 0-2: zero for absent/incorrect; one for a partly correct answer or a correct number without requested reasoning; two for a correct answer with the relevant argument. **Keep the original three-question threshold: questions 1-3 need at least 5/6. The full five-question quiz also needs at least 8/10.** Complete assigned study, the full practical assignment and the unaided transfer task before marking the lesson complete. Code-checker success alone is not completion.

The five module-review questions use the same two-point scale; suggested pass is 8/10. Correct missed work and solve a changed example. For the 15 written problems in each module, retain an independent attempt and a correction; proofs must identify assumptions and justified steps. At least two derivations/proofs per module must be reviewed, not merely checked for a final number.

The Gate B project uses the original 100-point rubric: correctness/requirements 30; tests/verification 25; reasoning/tradeoffs 15; quantitative error/performance analysis 10; reproducibility 10; communication/provenance 10. Suggested pass is 80/100, every essential check completed, and no unresolved critical defect in the claimed scope. A reviewer may accept another technically sound method when its assumptions and evidence are explicit.

Reference answers are study aids after an attempt. Explain all assisted code, record help received, and do the transfer task without following a supplied solution. Nothing here grants an academic credential. Passing a few numerical examples does not prove a mathematical theorem or demonstrate all of a semester's subject depth.